

UNIVERSITY DATABASE PROJECT REPORT

HOTEL & BUS BOOKING SYSTEM

An Integrated Relational Database Architecture for Accommodation and Schedule-Based Transportation Recommendation

Course Name: [Course Name Placeholder, e.g., CS302 Database Management Systems]

Department / Faculty: [Faculty of Computer Science / Department of Information Systems]

University: [University Name Placeholder]

Academic Year: [Academic Year Placeholder, e.g., 2025/2026]

Prepared By:

John Emil Ramzy

Instructor / Supervisor: [Instructor Name Placeholder]

Date of Submission: [Submission Date]

2. TABLE OF CONTENTS

1. Cover Page	1
2. Table of Contents	2
3. Abstract	2
4. Introduction	3
5. Problem Statement	3
6. Project Objectives	3
7. Project Scope	4
8. List of Different Project Implementations in the Chosen Field	4
9. Comparison Between Different Techniques in This Project	5
10. Chosen Project Design	7
11. System Workflow	7
12. Main System Components	8
13. Database Role in the System	8
14. Business Rules	9
15. Limitations	9
16. Future Enhancements	10
17. Conclusion	10
18. References	10

3. ABSTRACT

Modern travel planning often requires consumers to navigate disparate information systems to coordinate lodging and intercity transportation. This fragmented architecture introduces friction, schedule mismatches, and data inconsistency across reservations. This project presents the design and implementation of a centralized, relational database-driven **Hotel & Bus Booking System**. The primary objective is to facilitate seamless hotel room searches and reservations while providing an automated, schedule-aware bus transportation matching engine.

Under this database architecture, hotel reservations and transportation bookings remain strictly independent and optional; users are not compelled to reserve transportation when booking a room. However, upon preparing a hotel reservation, the relational engine analyzes the geographic proximity of nearby bus stops, hotel check-in/check-out timestamps, and active bus route schedules. It then query-matches and recommends suitable outgoing bus trips arriving prior to hotel check-in and return bus trips departing after check-out. By enforcing strict relational integrity, domain constraints, and high-order normalization, the proposed schema eliminates operational anomalies, prevents double-booking across rooms and seats, and offers an extensible foundation for integrated travel management applications.

4. INTRODUCTION

The hospitality and travel industries rely heavily on electronic reservation systems to manage inventory, process customer transactions, and maintain temporal schedules. Historically, accommodation platforms and passenger transit networks operated on isolated database infrastructures. Accommodation platforms prioritize date-range room availability, guest capacity, and check-in/check-out logistics. Conversely, transportation platforms focus on fixed-route transit, trip scheduling, vehicle capacity, and point-to-point stop sequences.

As traveler expectations evolve toward integrated travel planning, structured information management becomes essential. Managing accommodation and transportation under a unified database model enables cross-domain schedule reconciliation. Aligning intercity bus arrival and departure schedules with hotel check-in and check-out windows eliminates the manual burden of calculating transfer buffers and stop locations. Database systems serve as the core foundational layer in this environment, enforcing transactional consistency (ACID properties), preserving referential integrity across entities, and enabling multi-table joins to query spatial-temporal dependencies.

5. PROBLEM STATEMENT

Travelers routinely encounter operational friction due to the siloed nature of hotel and bus reservation systems. The central problems addressed by this project include:

- **Information Silos:** Hotel accommodation details and intercity bus schedules are managed in disconnected systems, requiring travelers to manually cross-reference arrival times, check-in policies, and geographical distances between transit hubs and lodgings.
- **Schedule Mismatches:** Failure to align bus schedules with check-in/check-out constraints often leaves travelers arriving hours before check-in or departing long after room check-out without convenient transportation options.
- **Inventory Integrity Issues:** Naive database implementations frequently suffer from concurrency anomalies, leading to double-booked hotel rooms or bus seats.
- **Data Redundancy & Inconsistency:** Storing location, stop, or schedule data redundantly across tables without strict normalization leads to update anomalies and data degradation over time.

6. PROJECT OBJECTIVES

The primary technical and structural objectives of this database project are:

1. **Relational Database Architecture:** Design a fully normalized relational schema (3NF) that models hotels, rooms, buses, seats, stops, routes, trips, and reservations.
2. **Decoupled Optional Bookings:** Ensure hotel bookings and bus trip reservations remain independent entities while supporting logical connections when requested by the user.
3. **Spatial-Temporal Transportation Matching:** Develop query-level logic that correlates hotel location (via nearby bus stops) and reservation dates/times with available bus trips.
4. **Data Integrity & Concurrency Enforcement:** Enforce strict foreign key constraints, unique keys, and transactional checks to guarantee that no room or bus seat is double-booked for overlapping time ranges.

7. PROJECT SCOPE

To ensure clear system boundaries, the functional scope of this database project is defined below:

Included Functionality:

- **User Management:** Customer account storage, authentication attributes, and contact records.
- **Hotel & Room Management:** Hotel metadata, location linkage, room types, room capacities, and night rates.
- **Bus Infrastructure Management:** Bus fleet records, individual seat layouts, bus stops, multi-stop routes, and scheduled bus trips.
- **Reservation Systems:** Independent processing of hotel room reservations and optional bus trip bookings.
- **Automated Trip Matching Engine:** SQL query logic identifying suitable outgoing/return bus trips based on hotel check-in/out schedules and geographical bus stop mapping.
- **Payment Record Tracking:** Basic transactional status and payment logging associated with bookings.

Excluded Functionality (Out of Scope for Current Version):

- Real-time vehicle GPS tracking, live telemetry, or dynamic traffic delay calculations.
- Third-party API integrations with external commercial bus providers or live payment gateways.
- Integration with non-bus transportation modes (e.g., commercial airlines, passenger rail networks).
- Dynamic pricing algorithms, yield management models, or machine-learning-based personalized recommendation systems.

8. LIST OF PROJECT IMPLEMENTATIONS IN THE CHOSEN FIELD

The travel and hospitality reservation field encompasses various architectural implementations depending on business requirements:

A. Hotel-Only Reservation Systems

Concept & Workflow: Focuses exclusively on room inventory, room types, pricing tiers, and calendar bookings. The workflow involves searching by city/date, selecting a room, and confirming stay dates.

Advantages & Limitations: Simple data domain and high performance for lodging. However, it completely ignores customer transit requirements.

B. Transportation-Only Reservation Systems

Concept & Workflow: Dedicated to vehicle routing, seat allocation, departure/arrival timetables, and passenger ticketing. Users select origin/destination hubs and departure times.

Advantages & Limitations: Highly optimized for transit scheduling and seat booking, but lacks awareness of customer destination lodging or schedule constraints.

C. Integrated Travel Reservation Systems (Selected Paradigm)

Concept & Workflow: Combines accommodation management with transportation booking. Users book a hotel and are presented with schedule-aligned transportation options within a unified interface.

Advantages & Limitations: Significantly improves user convenience and reduces scheduling errors. Requires a sophisticated relational schema with spatial-temporal query logic.

9. COMPARISON BETWEEN DIFFERENT TECHNIQUES IN THIS PROJECT

Evaluating architectural trade-offs is essential to establishing a robust database design. Below is an objective analysis of key technical alternatives.

9.1 Hotel Availability: Boolean Field vs. Date-Range Booking Records

Criteria	Technique A: Boolean Field (is_available)	Technique B: Date-Range Records (Selected)
Data Accuracy	Low; reflects only single static state (e.g., current moment).	High; calculated dynamically from reservation date ranges.
Historical Data	Overwritten upon state change; no audit history.	Fully preserved in historical reservation records.
Future Bookings	Cannot support multiple future reservations for one room.	Supports unlimited future non-overlapping date ranges.
Conflict Prevention	Requires manual system flags; highly prone to race conditions.	Enforced via relational overlap queries ('NOT EXISTS').
Scalability	Poor functional scalability for advanced booking engines.	Highly scalable; indexed on '(room_id, check_in, check_out)'.

Selection Rationale: Technique B was selected because a static Boolean flag cannot represent continuous temporal availability. Date-range records allow dynamic evaluation of availability across arbitrary future calendars.

9.2 Bus Route & Trip Modeling: Monolithic Schedule vs. Separated Entities

Criteria	Technique A: Route & Schedule Combined	Technique B: BusRoute & BusTrip Separated
Normalization	Denormalized; repeats route origin/destination on every trip.	3NF compliant; route geometry defined once, referenced by trips.
Schedule Flexibility	Rigid; route modification requires updating all schedule rows.	Flexible; trips represent instance executions of static routes.
Data Redundancy	High redundancy of stop lists and distance metrics.	Zero redundancy; route parameters shared across all trips.

Selection Rationale: Technique B decouples the invariant geographical route topology from instance-specific temporal executions (trips), adhering to normalization principles.

9.3 Bus Seat Management: Total Capacity vs. Individual Seat Modeling

Criteria	Technique A: Numeric Seat Capacity Counter	Technique B: Individual BusSeat Entities
Granularity	Coarse; tracks only total remaining count (e.g., 42 available).	Fine-grained; models specific seats (e.g., Seat 12A, Window).
Reservation Control	Decrements counter; cannot assign specific physical seats.	Maps reservations to individual 'seat_id' records per trip.
Integrity Constraints	Vulnerable to counter underflow/overflow anomalies.	Unique constraint on '(trip_id, seat_id)' prevents double booking.

9.4 Hotel-to-Bus Relationship: Direct Mapping vs. Network Topology

Modeling transportation connections to hotels can be approached directly or transitively:

- **Technique A (Direct Foreign Key: `Hotel → Bus`):** Hardcodes specific buses to specific hotels. This invalidates real-world transit behavior where a single bus trip serves multiple stops and nearby facilities.
- **Technique B (Network Mapping: `Hotel → BusStop → BusRoute → BusTrip`):** Connects hotels to nearby physical `BusStop` nodes. Bus routes consist of ordered sequences of stops. This allows a single bus trip to service multiple nearby hotels dynamically.

9.5 Booking Architecture: Decoupled Tables vs. Unified Parent Booking

Criteria	Technique A: Separate Hotel & Bus Tables	Technique B: Parent Supertype Booking Table
Schema Complexity	Moderate; independent schemas with clean isolation.	High; requires polymorphic foreign keys or sub-tables.
Query Performance	Optimal for independent hotel or bus operations.	Requires complex multi-table joins across supertypes.
Domain Independence	Strong; hotel booking operational without bus record.	Tightly coupled; structural overhead for single-mode bookings.

9.6 Database Normalization: Normalized Relational (3NF) vs. Denormalized

A third normal form (3NF) design eliminates redundant attributes (e.g., duplicating hotel addresses in room tables or bus driver details in seat bookings). While denormalization can accelerate specific read queries by eliminating joins, 3NF ensures absolute update consistency, crucial for transaction-heavy booking platforms where inventory overbooking must be prevented.

9.7 Database Paradigm: Relational Database (RDBMS) vs. Document NoSQL

Criteria	Relational Database (SQL - Selected)	Document Store (NoSQL)
Data Structure	Strict tabular schema with explicit primary/foreign keys.	Flexible nested JSON documents without fixed schema.
Referential Integrity	Enforced natively by engine via Foreign Key constraints.	Must be maintained manually in application-level code.
ACID Transactions	Native multi-table atomic transactions guaranteed.	Typically document-level; complex across distributed nodes.
Complex Joins	Highly optimized declarative SQL joins (`JOIN`).	Requires multiple queries or manual data aggregation.

RDBMS Justification: This project's core requirements—strict schedule constraints, relational join queries across transit nodes, and guarantee of non-overlapping bookings—align perfectly with relational database capabilities.

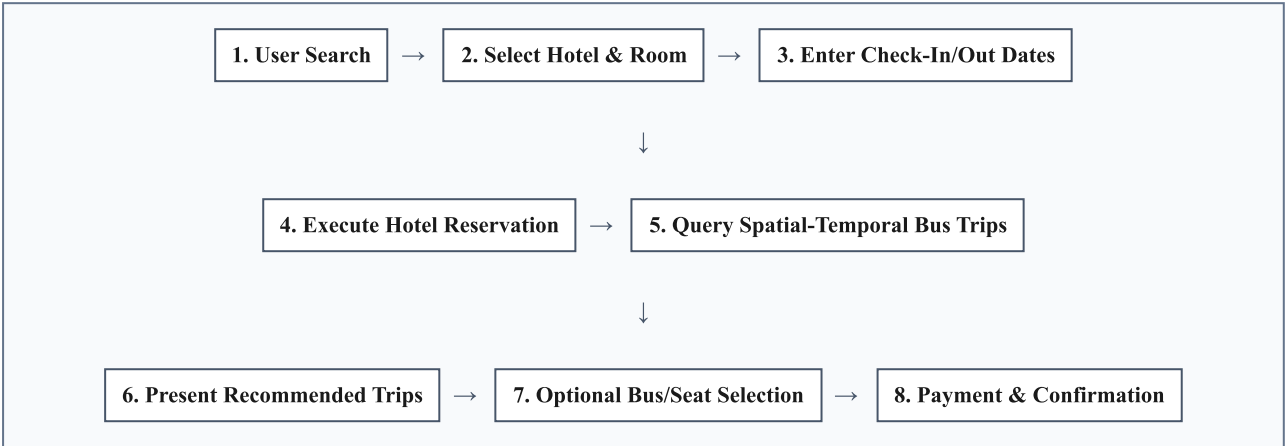
10. CHOSEN PROJECT DESIGN

The finalized system architecture utilizes a fully normalized (3NF) Relational Database Management System. The physical schema decouples operational entities to maintain high cohesion and low coupling:

- **Separation of Hotel and Room:** `Hotel` stores static entity metadata (name, address, geographic location), while `Room` represents individual rentable inventory units linked via foreign key.
- **Separation of Bus and BusSeat:** `Bus` holds physical vehicle properties (plate number, model), whereas `BusSeat` models individual seat identifiers and attributes.
- **Separation of BusRoute and BusTrip:** `BusRoute` defines the spatial path (origin stop, destination stop, total distance), while `BusTrip` models scheduled temporal executions with specific departure and arrival timestamps.
- **Geographical Linking via BusStop:** Hotels are associated with nearby `BusStop` entities through a junction table (`Hotel\_BusStop`), specifying distance or walking time.
- **Optionality & Matching Engine:** `HotelBooking` and `BusBooking` are separate transaction tables. The transportation matching engine executes an ad-hoc query correlating `HotelBooking.check\_in` and `check\_out` with `BusTrip.departure\_time` and `arrival\_time` for routes servicing the hotel's linked `BusStop`.

11. SYSTEM WORKFLOW

The complete conceptual execution flow for a user interacting with the integrated platform is illustrated below:



Workflow Detail: The user begins by searching for lodgings based on location and dates. After selecting an available room and confirming the hotel reservation, the system invokes the transit matching engine. If suitable bus trips exist that arrive prior to check-in and depart after check-out, they are presented to the user. The user may select one, both, or neither of the bus trips before proceeding to payment finalization.

12. MAIN SYSTEM COMPONENTS

The architectural scope of the system is partitioned into twelve logical functional modules:

1. **User Management:** Handles customer profile storage, authentication credentials, and user contact details.
2. **Hotel Management:** Maintains hotel administrative attributes, contact details, and location identifiers.
3. **Room Management:** Controls room inventory, room classification, occupancy limits, and nightly pricing.
4. **Hotel Booking:** Processes lodging reservation transactions, calculating total stay costs and locking room dates.
5. **Bus Management:** Registers physical bus vehicles, total capacity metrics, and operational statuses.
6. **Bus Seat Management:** Maps physical seat layouts (e.g., seat numbers, aisle/window designations) per vehicle.
7. **Bus Stop Management:** Catalogs physical transit stops, station names, and municipal location codes.
8. **Bus Route Management:** Defines ordered sequences of stops connecting origin and destination hubs.
9. **Bus Trip Management:** Schedules specific vehicle departures, assigning buses to routes at specific timestamps.
10. **Bus Booking:** Manages individual passenger seat reservations on specific scheduled bus trips.
11. **Transportation Matching Engine:** SQL query service calculating schedule suitability between stays and trips.
12. **Payment Management:** Logs transaction records, payment statuses, and billing references for bookings.

13. DATABASE ROLE IN THE SYSTEM

The relational database serves as the authoritative state store and logic enforcer for the platform:

- **Data Persistence:** Ensures all system entities (users, lodgings, transit schedules, receipts) are durably stored across system restarts.
- **Relationship Enforcement:** Maintains strict structural dependencies via primary key-foreign key pairs across all entity modules.
- **Transactional Integrity (ACID):** Guarantees that multi-step operations (such as reserving a room and two bus seats simultaneously) execute completely or roll back entirely in case of failure.
- **Double-Booking Prevention:** Utilizes relational constraints and isolation levels to prevent simultaneous booking of identical room dates or bus seats by competing transactions.
- **Complex Spatial-Temporal Querying:** Enables high-performance multi-table SQL joins that match hotel geographical locations and stay durations with bus trip timetables in real time.

14. BUSINESS RULES

The system logic is governed by explicit business rules enforced at both the database and application levels:

- **Hotel & Room Rules:** A hotel may contain multiple rooms, but each room belongs to exactly one hotel. A room cannot have overlapping confirmed reservation date ranges:
`(Existing.CheckIn < New.CheckOut) AND (Existing.CheckOut > New.CheckIn) = FALSE`
- **Bus & Trip Rules:** A bus contains multiple physical seats. A bus trip belongs to a single route and utilizes one assigned bus. An individual bus seat cannot be booked more than once for the same trip instance.
- **Transit Linkage Rules:** A hotel can be linked to one or more nearby bus stops with explicit proximity metrics.
- **Independence Rule:** Hotel reservations do not require bus bookings. Bus trip selection is strictly optional.
- **Schedule Buffer Rules:** An outgoing bus trip is classified as "suitable" only if its arrival time precedes hotel check-in time by at least a predefined safety buffer (e.g., 60 minutes). A return bus trip is suitable only if its departure time follows hotel check-out time by at least the safety buffer.
- **Payment Rule:** Each payment record must reference a valid hotel booking, bus booking, or unified transaction reference.

15. LIMITATIONS

In accordance with academic standards, the structural limitations of the current implementation are explicitly acknowledged:

- **Static Transit Schedules:** Bus schedules and routes rely on internal stored database tables rather than live API streams from external transport authorities.
- **Absence of Real-Time GPS:** The system cannot dynamically adjust recommended trips based on live vehicle tracking, road congestion, or transit delays.
- **Geographic Simplification:** Proximity between hotels and bus stops is calculated using static table mappings or simplified Euclidean distance rather than full GIS road-network routing.
- **Mock Financial Transactions:** Payment tracking is limited to internal transactional logging; live third-party financial gateways (e.g., Stripe, PayPal) are not integrated.
- **Single Transit Mode:** The current matching engine supports intercity buses only, excluding commercial passenger airlines or rail systems.

16. FUTURE ENHANCEMENTS

The modular relational architecture established in this project provides a foundation for several planned future extensions:

- **Dynamic GIS Integration:** Incorporate PostGIS spatial database extensions to perform exact geographic radius searches and calculate true walking/driving paths between hotels and bus stops.
- **Multi-Modal Transit Support:** Extend transit route entities to accommodate passenger trains and regional flights alongside bus networks.
- **External Provider API Connectors:** Develop synchronization services to import real-time transit schedules and live seat availability from third-party transport operators.
- **Advanced Pricing & Promotion Engine:** Introduce dynamic seasonal pricing algorithms, promotional discount codes, and user loyalty points tracking.
- **Mobile Application Interface:** Build native mobile client applications utilizing push notifications for check-in alerts and transit boarding updates.

17. CONCLUSION

This project successfully demonstrates the design and theoretical foundation of an integrated relational database for a **Hotel & Bus Booking System**. By bridging the gap between lodging platforms and transportation networks, the proposed architecture effectively solves the schedule mismatch problem common in travel planning.

Crucially, the system preserves operational flexibility by maintaining strict independence between hotel bookings and transit reservations while offering automated, schedule-aware trip recommendations when requested. Through rigorous 3NF normalization, foreign key constraint enforcement, and formal business rule definitions, the database guarantees data consistency, eliminates inventory double-booking, and provides a robust, extensible foundation for real-world travel information systems.

18. REFERENCES

1. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
2. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill Education.
3. Date, C. J. (2003). *An Introduction to Database Systems* (8th ed.). Addison-Wesley.
4. Garcia-Molina, H., Ullman, J. D., & Widom, J. (2008). *Database Systems: The Complete Book* (2nd ed.). Pearson.
5. Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a Database System. *Foundations and Trends in Databases*, 1(2), 141-259.

